JAX Flight Simulator

What the project is, what it proves, and what it does not

A plain-English summary

7 August 2026

This is a flight simulator written to do one specific job: find out how a large aircraft responds when it flies into clear-air turbulence. It is not a game and it has no scenery. It is a physics engine with an instrument panel bolted on, so that a person can fly the aircraft into a turbulence pattern measured from a real incident and watch what happens.

The project has an unusual rule that shapes everything in it: every number must come from a published source, and must say which one. Where no source exists, the gap is written down rather than filled with a plausible guess. That rule is why this document can cite every figure in it.

> **The two rules the project runs on**
>
> 1. Flag, never invent. Every number carries the table it came from. A parameter no source supplies is named as a declared modelling choice, not given a plausible default.
>
> 2. Never edit a tolerance to make a test pass. If a validated measurement moves, something real broke.

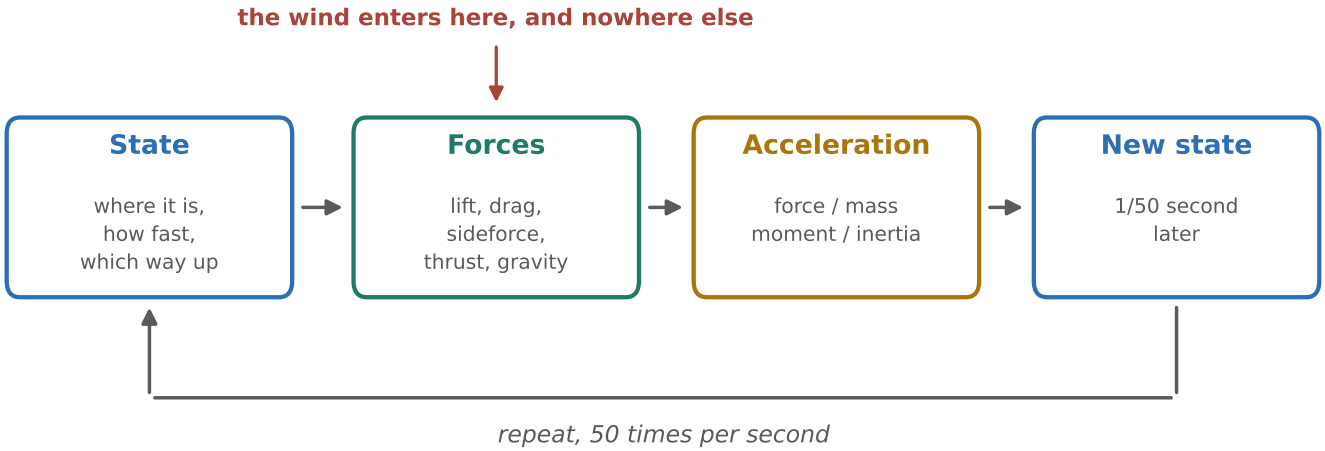
Language	Python 3.10 with JAX (a numerical library that compiles maths to fast code)
Size	About 4,300 lines of code and 256 automated tests
Aircraft	Boeing 747, Piper PA-28-180 Cherokee, Cessna 172
Physics	Six degrees of freedom, rigid body, fixed-step Runge-Kutta integration
Purpose	Clear-air turbulence response, compared against two published papers
Status	Flight model validated; turbulence layer partly built

Every figure in this document is marked with a source tag such as [S1] or [M3].
The full list is on the last page.

What a flight simulator actually has to do

An aircraft in flight is a solid object being pushed around by three things: the air flowing over it, the thrust of its engines, and gravity. If you know where it is, how fast it is going and which way it is pointing, you can work out those forces. From the forces you get the acceleration. From the acceleration you get the next position and attitude, a fraction of a second later. Then you do it again.

That loop is the whole simulator. This one runs it fifty times a second.

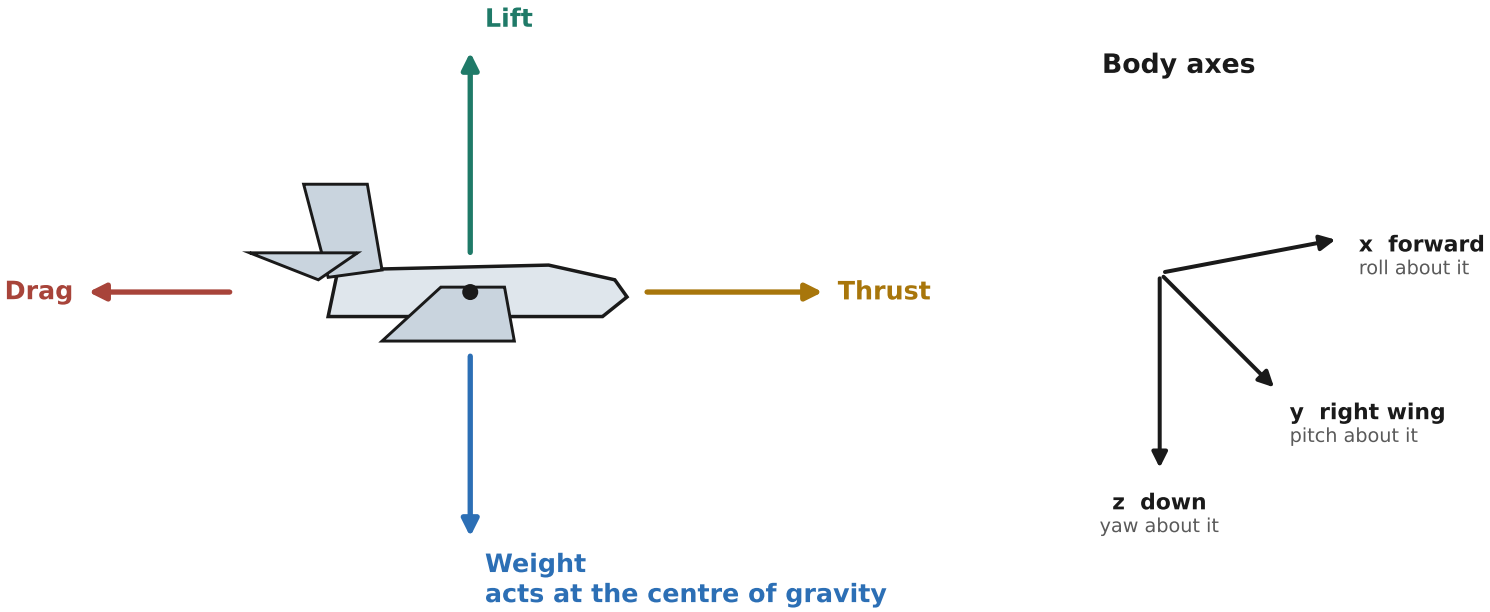


"Six degrees of freedom" is the standard phrase for tracking all the ways a rigid body can move: three of position (north, east, down) and three of rotation (roll, pitch, yaw). Simpler simulators fix some of these. This one does not, which is why an aircraft in it can be upset in a way that couples roll into yaw and back, exactly as a real one is.

> **Why the wind only enters in one place**
>
> Turbulence changes the air the wings are flying through. It does not reach out and shove the aeroplane. So the wind is subtracted from the aircraft's velocity before the aerodynamic forces are computed, and is kept out of everything else. Putting it anywhere else is a classic modelling error, and this project has a test that fails if anyone does.

The step size never changes. A variable step would make a run impossible to reproduce exactly, and would leave the accuracy of the answer at the mercy of whatever else the computer happened to be doing.

The forces, and the axes they act in



Everything is measured in axes fixed to the aircraft: x out of the nose, y out of the right wing, z down through the floor. Rotating about those three gives roll, pitch and yaw. The aircraft's position is tracked separately in north-east-down axes fixed to the ground.

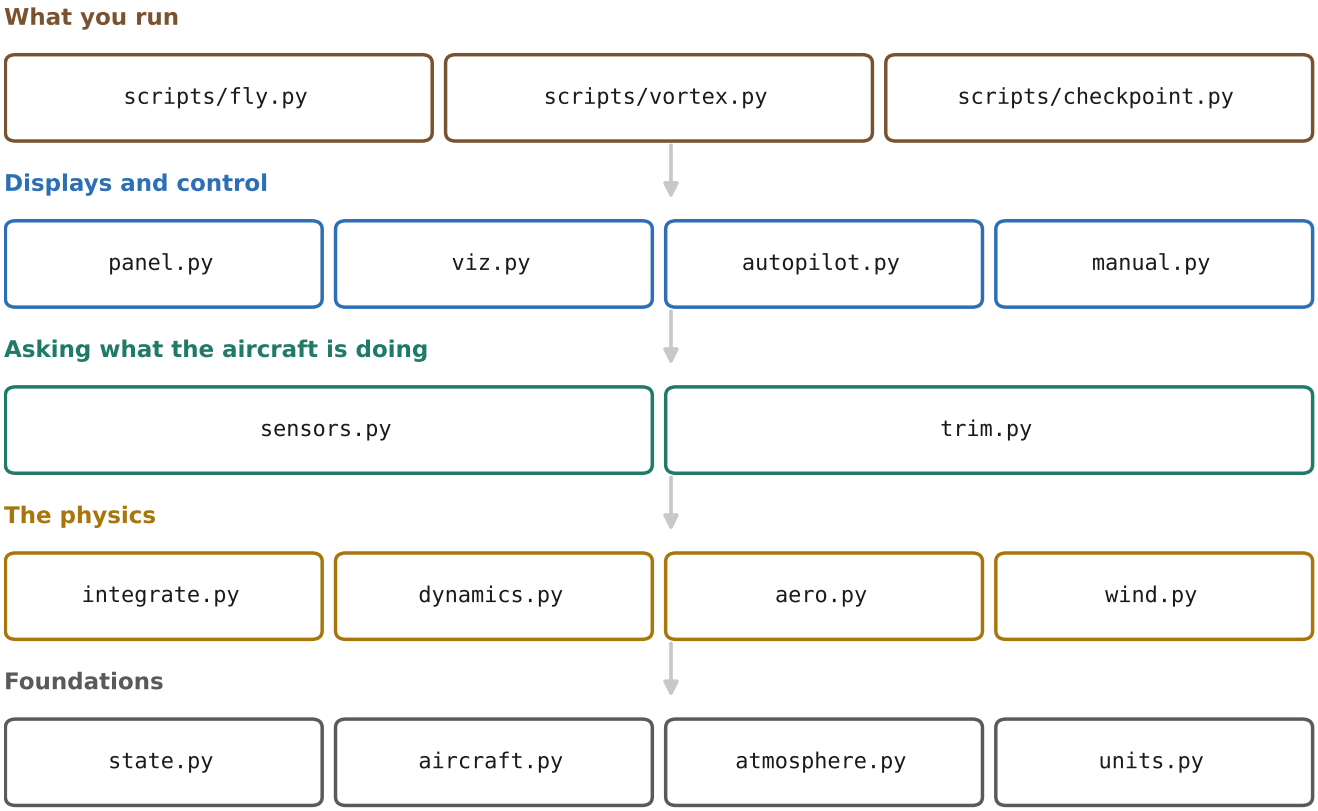
The aerodynamic forces come from a standard build-up: each force and moment is a sum of contributions from angle of attack, from the rotation rates, and from the control surfaces, each multiplied by a coefficient measured in a wind tunnel or in flight. Those coefficients are the aircraft. They are what the source documents supply, and they are what makes a Boeing 747 behave like a Boeing 747 rather than like a scaled-up Cessna.

> **Angle of attack, in one sentence**
>
> The angle between where the nose is pointing and where the aircraft is actually going through the air — it is what produces lift, and a gust changes it instantly without the aircraft moving at all.

Orientation is stored as a quaternion, a four-number way of representing a rotation. The alternative — three angles — breaks down when the aircraft points straight up or straight down. Quaternions do not, and the project has a test confirming the representation stays valid over one hundred thousand steps [M1].

How the code is put together

Fourteen files, each with one job. The arrows show what depends on what: a file only ever uses the ones below it, which is what stops a change in the instrument panel from being able to affect the physics.



File	What it does
`units.py`	Conversion constants only. Feet to metres and so on, in one place.
`state.py`	What the aircraft's state is, and the quaternion maths.
`atmosphere.py`	Air density and temperature against altitude, to 20 km.
`aero.py`	Turns motion into aerodynamic forces. Never sees ground speed.
`dynamics.py`	Newton's laws for a rigid body. The wind enters here.
`wind.py`	The turbulence patterns themselves.
`integrate.py`	Takes one time step, accurately.
`aircraft.py`	The three aircraft, every number citing its source table.
`sensors.py`	The only supported way to ask what the aircraft is doing.
`trim.py`	Solves for the controls that hold steady level flight.
`autopilot.py`	Holds a height, a heading and a speed.
`manual.py`	Hand flying, trim, and the switch between the two.
`panel.py`	The live instrument panel.
`viz.py`	Saves a flight, and draws the after-the-fact plots.

The one rule worth knowing

Ask what the aircraft is doing through sensors.py, never by reaching into the state directly. The reason is on page 7.

The three aircraft, and their sources

An aircraft in this simulator is a list of about forty numbers. Every one of them is transcribed from a published document, with a comment naming the table it came from. Where a source is missing a number, the number is set to zero and the gap is recorded — it is never guessed.

Aircraft	Source	Standing
Boeing 747	NASA CR-2144, section IX [S1]	Fully validated. The only aircraft used for turbulence work.
Piper PA-28-180 Cherokee	McCormick, via a worked example [S4]	The validated light aircraft. No second source for its lateral numbers.
Cessna 172	Roskam / USAF DATCOM, via PyFME [S5]	OUT OF SCOPE. Its rudder data is missing, so its turns are wrong. No result may be quoted from it.

Cruise conditions used, computed from the code

Boeing 747: 235.9 m/s true airspeed at 12192 m (40000 ft), about Mach 0.80
Cherokee: 50.0 m/s at 1500 m
Cessna 172: 60.0 m/s at 1524 m

Does the 747 actually behave like a 747? The way to check is to compare its natural oscillations against the ones the source document measured. An aircraft disturbed in flight wobbles in characteristic ways, each with its own period and damping, and those are a fingerprint of the aerodynamics.

Motion	This model	CR-2144 [S1]	Difference
Dutch roll frequency	0.943	0.947 rad/s	0.4%
Dutch roll damping	0.0361	0.0349	3.4%
Roll response time	1.795 s	1.779 s	0.9%
Spiral time	138.0 s	137.0 s	0.8%
Phugoid frequency	0.0553	0.0673 rad/s	17.8% explained
Short-period damping	0.3425	0.387	11.5% explained

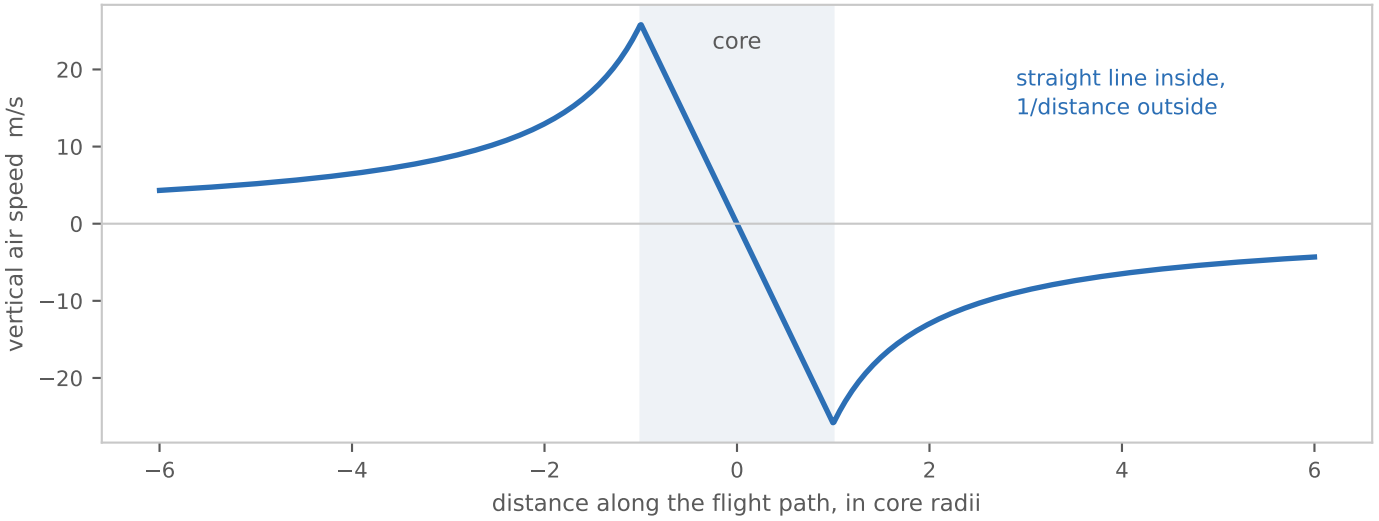
The four lateral numbers agree to within one percent. The last two do not, and the project says why rather than hiding it: the simulator deliberately leaves out five of the source's coefficients, and a second model built with them restored closes both to about one percent [M2]. That is an understood, documented omission, not an unexplained error.

The turbulence: a real one, from a real incident

In 1985 four NASA researchers took the flight recorders from airliners that had hit severe clear-air turbulence near the top of the troposphere, and worked backwards to reconstruct the air the aircraft had flown through. What they found was not random buffeting. It was a row of large, organised spinning vortices [S2].

Each vortex is modelled as a spinning core with smooth circulation around it — a rotating solid centre embedded in an irrotational flow. Inside the core the air speed rises in a straight line from the middle outwards. Outside, it falls away as one over the distance.

One vortex, Hannibal case: peak vertical air speed 25.8 m/s [computed]

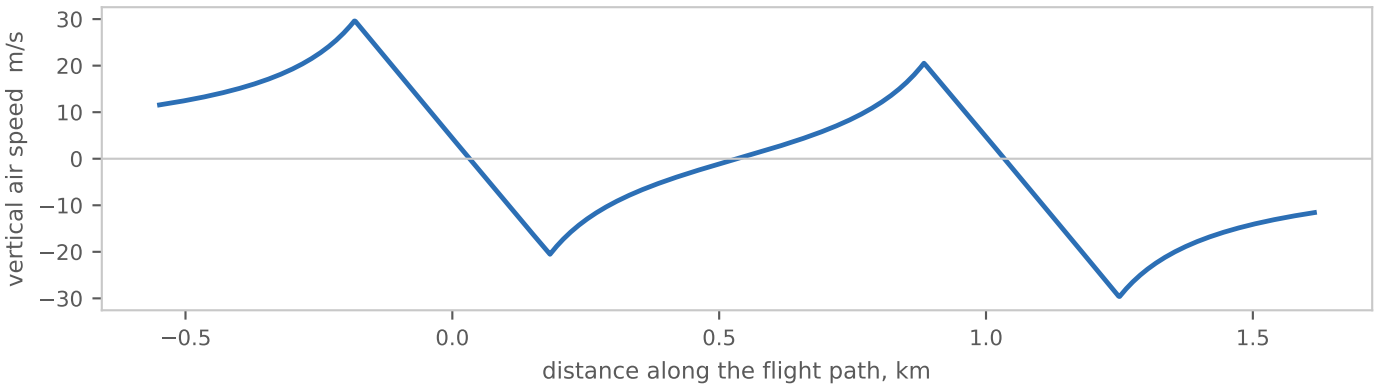


Case [S2]	Altitude	Core radius	Edge speed	Spacing
Hannibal, Missouri	37,000 ft	183 m (600 ft)	25.9 m/s (85 ft/s)	1067 m
Morton, Wyoming	39,000 ft	137 m (450 ft)	21.3 m/s (70 ft/s)	975 m

Why the spacing is not a free parameter

The two cases give vortices spaced 2.92 and 3.56 core diameters apart. The paper checks that against Scorer's theoretical prediction of 2.7 for this kind of wave instability, and it agrees. That check is what turns the spacing from a number somebody chose into a number the physics requires. [S2]

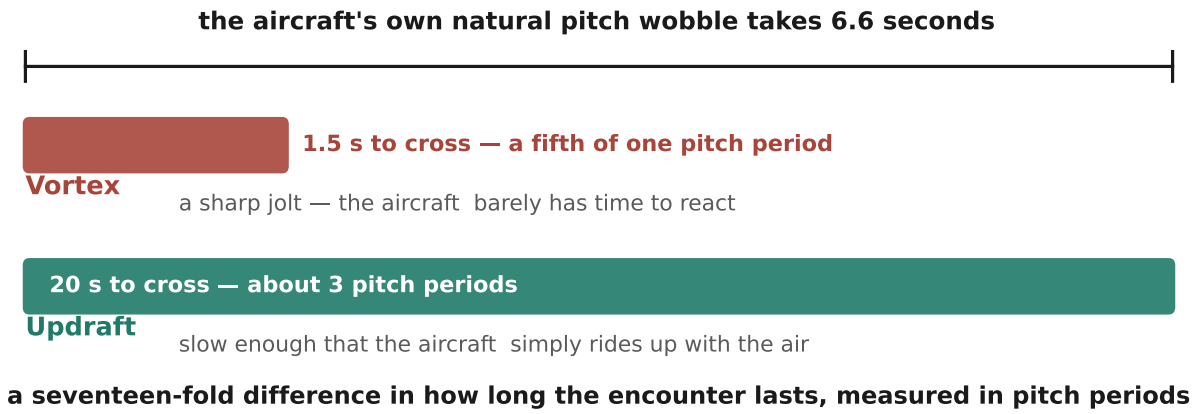
Two cores, 1067 m apart, as the aircraft flies through them [computed]



The main finding, and why it works

A second paper, from 1994, plotted many turbulence encounters as pitch change against g-load change, and found they fell into separate clusters depending on what caused them [S3]. Vortex encounters landed in one place; thunderstorm updrafts in another. The obvious question is why.

This project's answer is that it is purely a matter of timing, and needs no exotic aerodynamics at all.



The finding, in one sentence

A vortex is crossed in about a fifth of the aircraft's natural pitch period and hits it like an impulse; an updraft takes about three of them and the aircraft has time to follow it. That seventeen-fold separation is the whole explanation, and it is a rigid-body timing effect requiring no non-linear aerodynamics whatsoever.

Measured in this model [M3]	Value	What the paper reports
Gap between gusts, Hannibal	4.52 s	paper says “about 5 s apart” [S2]
Pitch change, first vortex core	2.24 deg	paper's figure shows about 1.4 deg [S3]
Pitch change, updraft column	4.37 deg	paper states 5.2 deg [S3]
Peak g-load swing, vortex	−1.23 g	paper's band is −1.7 to −2.0 g [S3]

The model reproduces the pattern and the ordering, and lands in the right region without matching any single number exactly. That is the honest claim, and the project makes only that claim: the papers never say what aircraft type they measured, and the two vortex cases were DC-10s rather than 747s, so a like-for-like comparison is not available at all.

One thing the model provably cannot do

Real encounters push harder downwards than upwards. Both papers put that down to the wing stalling. This model's lift rises in a perfectly straight line with angle of attack and never stalls, so an up-gust and an equal down-gust must give exactly equal and opposite loads. Reproducing the asymmetry would need stall data for the 747 that no source held by the project contains. It is recorded as impossible rather than pending.

The mistake that hid in plain sight

This is worth a page because it is the most instructive thing in the project, and because the fix shaped the design of everything after it.

An aircraft has two different speeds. Its speed over the ground, and its speed through the air. In still air they are identical. In wind they are not, and it is the speed through the air that keeps an aeroplane flying — which is why an aircraft can be stationary over the ground in a strong enough headwind and still be flying perfectly well.

In still air — the two are the same number

In a 25 m/s headwind — they are not

Only the lower number keeps the aircraft flying. The simulator was using the upper one.

Two parts of the project asked for “airspeed” and were quietly handed the ground speed instead: the autopilot, and the display that computes angle of attack. In still air this is invisible, because the two numbers are the same. Every one of the two hundred and nine tests in the project at the time was a still-air test, so not one of them could see it.

> **How large the error actually was**
>
> In a vortex encounter, the angle of attack computed the wrong way differed from the correct one by up to 7 degrees — measured, not estimated. The correlation between g-load and angle of attack was 0.9990 computed correctly and 0.5572 computed wrongly. The wrong version would have destroyed the main result. [M4]

The fix was deliberately heavy-handed. A single module now owns the question “what is the aircraft doing?”, it cannot be called without stating what the air is doing, and the raw ground velocity is simply not in scope for anything that might misuse it. There is no longer a wrong number available to reach for.

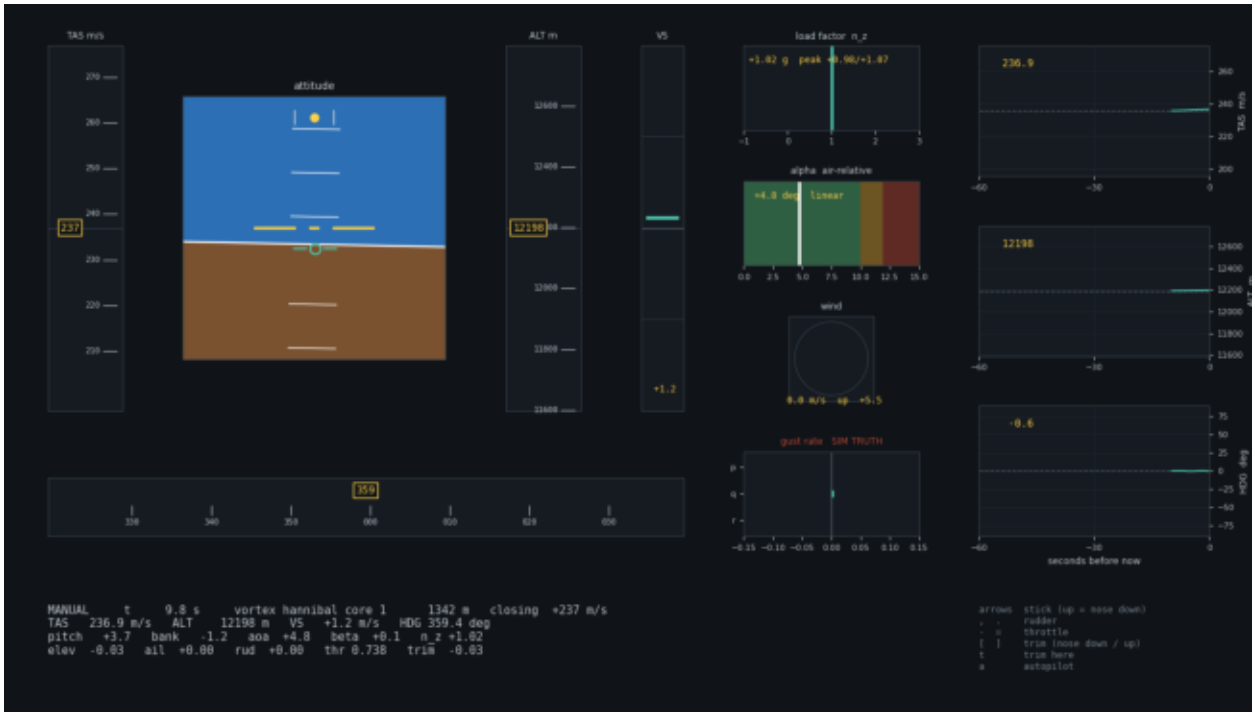
The lesson was written into the project's rules: any quantity that has both an air-relative and a ground-relative form needs at least one test that flies through real wind. A whole test file now exists for that and nothing else.

> **And the fix was checked by breaking it again**
>
> Both bugs were deliberately re-introduced to confirm that exactly the two tests written to catch them went red, and that nothing else moved. A test that has never been seen to fail is not yet evidence of anything.

The instrument panel

The panel follows the standard cockpit layout that the RAF standardised in 1937 and every aircraft has used since [S8]: attitude in the middle, airspeed to its left, altitude to its right, heading below. A pilot trained on one aircraft can read any other. On the right is a second set of instruments no real cockpit has, showing the quantities this project measures.

The 747 approaching the first vortex core — range counting down in the status block



Instrument	What it shows
Airspeed, altitude	Sliding scales, as in a modern airliner. The number stays still and the scale moves past it.
Attitude	Artificial horizon, with the aircraft symbol fixed and the world moving behind it.
Slip ball	Shows sideways force, not sideslip angle — a real ball is a pendulum, and the two are different quantities.
Vertical speed	Rate of climb or descent. Without it you cannot hold a height by hand; you end up chasing the altimeter.
Load factor	The g-load. This is the vertical axis of the comparison on page 6, and it holds the peak of any excursion.
Angle of attack	With a coloured band marking where this model's straight-line aerodynamics stop being trustworthy.
Wind and gust rate	The gust rotation is labelled SIM TRUTH, because no real instrument can measure it.

Flying it

Arrow keys are the control stick, comma and full stop the rudder, minus and equals the throttle, square brackets the trim, 't' trims out whatever you are holding, and 'a' hands over to the autopilot.

What is proven, and what is not

The project keeps a ledger. Every claim in it is a measured number with the tolerance its automated test enforces, and no row is ever deleted — if a figure changes, the old one stays on the record, superseded.

Check [M1, M5]	Measured	Test allows
Angular momentum held over 10 minutes	0.00000000000057	1 part in 10^11
Orientation stays valid over 100,000 steps	holds	1 part in 10^12
Coordinated turn against the textbook formula	1.12% off	under 3%
Zero-strength wind against no wind at all	bit-for-bit identical	exactly equal
Autopilot holds airspeed, not ground speed	yes	within 1.5 m/s
Angle-of-attack change from a 10 m/s updraft	2.43 deg	within 2%

What that fourth row is for

Running the simulator with a wind model that returns exactly zero wind gives results identical to the last decimal place to running it with no wind model at all. That proves the turbulence machinery adds nothing and disturbs nothing when it is switched off — so any difference seen with it switched on is real.

Set against that, the project is equally explicit about its limits:

- **Cannot reproduce the up/down asymmetry**
  The wing never stalls in this model, and no source held by the project has stall data for the 747. Recorded as impossible, not pending.
- **Cannot match the papers' g-loads exactly**
  Neither paper says what aircraft it measured, and the vortex cases were DC-10s. Only orderings and magnitudes are ever claimed.
- **The vortex numbers carry the source's own uncertainty**
  The paper derived them through an assumed aerodynamic model, so a 1-degree error becomes 4.12 m/s of wind. Treat them as accurate to roughly a quarter.
- **Any encounter past about 10-12 degrees of angle of attack**
  reports lift the sources say is not there. The panel now marks that band in red so a run that leaves the valid range says so.
- **The Cessna 172 is out of scope**
  Its rudder data is missing from the source, so its turns are wrong. It flies and passes its tests, but no result may be quoted from it.

Running it, and how fast it goes

Command	What it does
`python -m pytest flightsim/tests -q`	All 256 tests. The only complete statement of what works.
`python scripts/fly.py`	Fly it yourself, in still air.
`python scripts/fly.py --wind hannibal`	Fly into the 1985 vortex array.
`python scripts/vortex.py`	Run the turbulence analysis and draw the figure.
`python scripts/checkpoint.py`	Check the 747 against CR-2144.
`python scripts/analyse.py run.npz`	Re-plot a saved flight without re-flying it.

On the question of speed: the simulator deliberately does not use much of a modern computer. Two separate reasons, and only one of them is worth fixing.

Why the processor looks idle

The whole loop runs on a single processor core, and paces itself to the wall clock — it does exactly fifty physics steps per second and then waits. On a twelve-core machine one saturated core is about eight percent of the total, which is what the task manager shows. More cores cannot help: drawing is inherently sequential, and each physics calculation is far too small to split up.

The frame rate was another matter. Profiling found the physics was never the problem — it is a rounding error in the budget — and that most of the time was going somewhere unexpected. [M6]

Where the time went	Time	Note
One step of the physics	0.11 ms	0.7% of one core, at 50 steps a second
Reading the instruments BEFORE	6.56 ms	called once per physics step
Reading the accelerometers BEFORE	10.70 ms	called once per frame
Drawing the panel	30 ms	the genuine cost
Reading the instruments AFTER	0.03 ms	234x faster

The instrument reads were doing about twenty tiny calculations one at a time, and the overhead of launching each one dwarfed the arithmetic. Compiling them into a single operation — a two-line change — cut the frame from 73 ms to 38 ms and took the frame rate from 13.7 to 26.4 per second. Drawing the panel is now four-fifths of what remains, and the three sixty-second history strips, at 1,201 points each redrawn every frame, are the obvious next target.

Where the computer's full power would actually be used

Not in flying one aircraft. The project is built to run many simulated flights at once — the same turbulence encounter flown by hundreds of copies of the aircraft in parallel, to get error bars on the result. That is what the underlying library is for, and that is the step this work was building towards.

Sources

Every number in this document traces to one of the following. Source tags [S] are published documents; measurement tags [M] are figures produced by the project's own automated tests and recorded in its evidence ledger.

[S1] Heffley, R. K. and Jewell, W. F. (1972). Aircraft Handling Qualities Data. NASA CR-2144, section IX. Supplies the Boeing 747 geometry, mass, inertia and aerodynamic coefficients, and the reference oscillation table on page 5. Contains no stall data, which is why page 6's asymmetry is out of reach.

[S2] Parks, E. K., Wingrove, R. C., Bach, R. E. and Mehta, R. S. (1985). Identification of Vortex-Induced Clear Air Turbulence Using Airline Flight Records. Journal of Aircraft, volume 22, number 2, pages 124-129. Supplies the vortex model, both identified cases, and the spacing check against Scorer.

[S3] Wingrove, R. C. and Bach, R. E. (1994). Severe Turbulence and Maneuvering from Airline Flight Records. Journal of Aircraft, volume 31, number 4, pages 753-760. Supplies the updraft magnitudes and duration, the g-load statistics, and the cluster diagram discussed on page 6.

[S4] McCormick, B. W., via a worked example. Supplies the Piper PA-28-180 Cherokee coefficients. No second source was found for its lateral set.

[S5] Roskam, J. / USAF DATCOM, via the PyFME project. Supplies the Cessna 172 coefficients. Its rudder data is omitted and inconsistent, so the entire rudder set is zeroed and the aircraft is out of scope.

[S6] Nelson, R. C. / Etkin, B. / McRuer, D. Supply the Navion coefficients used in testing. No published oscillation table was found, so its tests assert ranges rather than values.

[S7] MIL-F-8785C. The military specification for the Dryden turbulence spectra. Not yet used: its intensity figure at altitude is a chart to be read off, not a formula, and has still to be digitised.

[S8] The standard six-instrument “basic T” arrangement, standardised by the Royal Air Force in 1937 and used on essentially every aircraft since; and the modern primary flight display layout that preserves it. General aviation practice rather than a single document.

Measurements

[M1] Integrator and rigid-body conservation checks. PROJECT.md section 4, “Integrator and rigid body”; tests in test_conservation.py.

[M2] Boeing 747 oscillations against CR-2144. PROJECT.md section 4, “747 modes vs CR-2144”; tests in test_cr2144_modes.py.

[M3] Vortex and updraft encounter results. PROJECT.md section 4, “Vortex and updraft encounters”; produced by scripts/vortex.py.

[M4] Air-relative sensing, including the 7-degree error and the two correlations. PROJECT.md sections 4 and 6; tests in test_sensors.py.

[M5] Air-relative sensing tolerances. PROJECT.md section 4, “Air-relative sensing”.

[M6] Live-loop timing, 7 August 2026, 12-core machine, medians of 120 runs.

The project's own standing record is docs/PROJECT.md, which carries the full evidence ledger, the list of known gaps, and the session-by-session history.